IRWA 2026 – Project Part 2: Indexing and Evaluation

Group and repository

Student	ID
Farith Godoy	U214405
Anna Vasileva	U255290

- **GitHub repository:** github.com/Mousemices/R-part1
- **TAG:** `IRWA-2026-part-2`
- **Code:** `project_progress/part_2/02_indexing_evaluation.ipynb`
- **Relevance labels for our test queries:** `project_progress/part_2/custom_query_labels.csv`

Overview

In this part we build an inverted index over the 28,080 products of the fashion dataset, answer conjunctive (AND) queries and rank the matches with TF-IDF and cosine similarity. We then implement seven evaluation metrics and apply them to the two validation queries and to five test queries that we labelled ourselves.

The notebook follows the order of the statement: section 0 reuses the Part 1 preprocessing, section 1 covers indexing and ranking, section 2 covers evaluation, and section 2.4 adds diagnostics used in the analysis.

1.1 Inverted index

The index is a Python dictionary that maps each stemmed term to the list of products (`pid`) that contain it: `{term: [pid_1, pid_2, ...]}`. It has **20,426 terms**. Each product is added once per term (we iterate over the set of its terms), so the length of a postings list is the document frequency of the term.

Preprocessing. It is the same as in Part 1: lowercase, alphanumeric tokens (`[a-z0-9]+`), English stop-word removal (NLTK) and Porter stemming. Queries go through the same function, so query terms and index terms always match (e.g. *jeans* → *jean*, *printed* → *print*).

Indexed fields. We index the terms of `title`, `description` and `product_details` (flattened into `key value` text). These three fields describe the product itself: type, colour, fabric, fit, sleeve, pattern.

- `product_details` is important for retrieval: words such as *denim*, *cotton* or *slim* often appear only there and not in the title.
- It also explains why the vocabulary grows from 4,902 terms (title + description, Part 1) to 20,426: style codes and model names add many rare terms.

Fields not indexed as text.

- `brand`, `seller`, `category` and `sub_category` are kept as metadata. `category` is almost constant (97% *Clothing and Accessories*), so it adds no discrimination, and seller names are not words a user types. They will be used as filters in Part 3.

- `selling_price`, `actual_price`, `discount`, `average_rating` and `out_of_stock` are numeric or boolean. They are stored as numbers (Part 1) and not indexed as terms, because a term such as *921* carries no meaning for a text query. They are meant for filtering and re-ranking.
- `pid` is kept unchanged as the document identifier used in the evaluation.

1.2 Conjunctive (AND) search

A product is returned only if it contains **all** the query terms, so the result is the intersection of the postings lists of the query terms.

1. The query is preprocessed with the same function as the documents.
2. We take the postings list of each distinct query term as a set and sort the sets from shortest to longest.
3. We intersect them one by one, starting from the shortest. The intermediate result can only shrink, so starting small keeps every intersection cheap, and we stop as soon as it is empty.

A query term that is not in the index gives an empty postings list, so the AND result is empty. For example, *men slim jeans blue* returns 222 products.

1.3 Test queries

We chose five clothing queries from the most frequent title terms (Figure 1), combined with more specific terms that test different parts of the system.

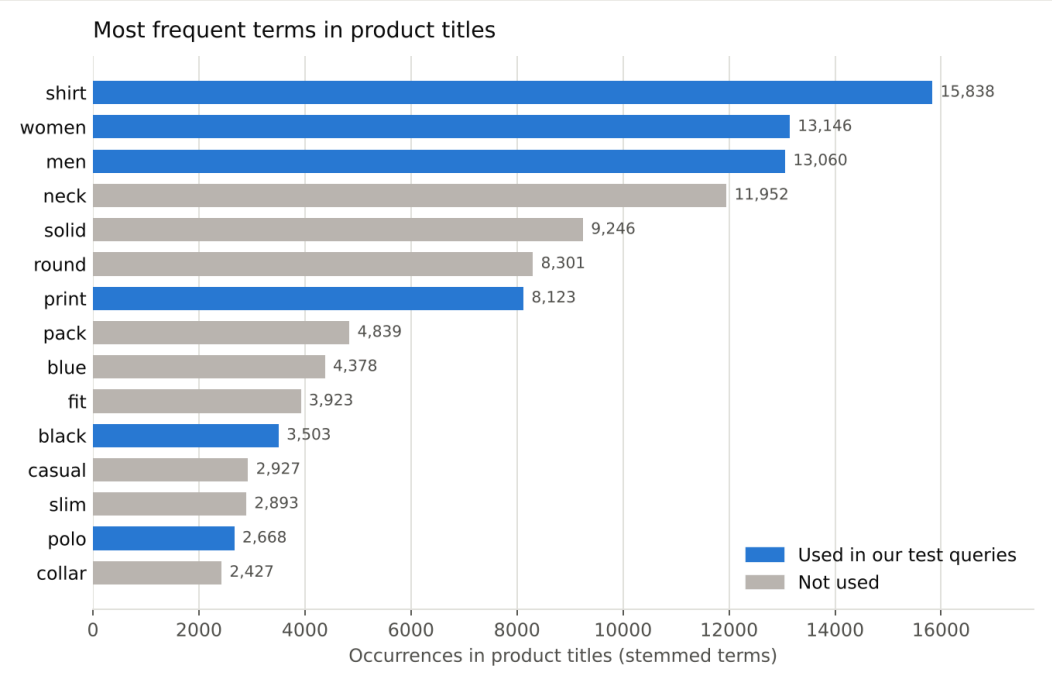


Figure 1. The 15 most frequent stemmed terms in product titles. Six of them (blue) are used in our test queries; the rest are generic words such as *neck*, *solid* or *round*.

Query	Stemmed terms	AND results	Why we chose it
men black polo shirt	men, black, polo, shirt	229	Only very popular terms: a common, easy search
women printed cotton kurta	women, print, cotton, kurta	42	A specific ethnic product with pattern and fabric
grey track pants	grey, track, pant	327	No gender: men's and women's products are both valid
men denim jacket	men, denim, jacket	73	*denim* is often only in the fabric field (product details)
women yoga pants	women, yoga, pant	19	A usage word that rarely appears in titles

1.4 Ranking with TF-IDF and cosine similarity

Only the products returned by the AND search are ranked. Each product and the query are represented as TF-IDF vectors and sorted by cosine similarity.

$$tf_{t,d} = \frac{n_{t,d}}{|d|} \qquad idf_t = \log \frac{N}{df_t} \qquad w_{t,d} = tf_{t,d} \cdot idf_t$$

$$\cos(q, d) = \frac{\sum_{t \in q} w_{t,q}\, w_{t,d}}{\|q\|\ \|d\|}$$

Here *n(t,d)* is the number of occurrences of *t* in product *d*, |*d*| the total number of terms of *d*, *N* = 28,080 and *df(t)* the length of the postings list of *t*.

Decisions.

- TF is normalised by the length of the product, so long descriptions do not get high weights just by repeating words.
- The query uses the same TF definition and the same idf as the products.
- TF, IDF, the TF-IDF vectors and the norm of every product vector are computed once and reused for every query.
- Only the query terms contribute to the dot product, so we only look up those terms in each product vector.

IDF behaves as expected: idf(*shirt*) = 0.515, while idf(*kurta*) = 3.575.

Top 3 per test query.

Query	#1 (score)	#2 (score)	#3 (score)
men black polo shirt	Solid Men Polo Neck Black T-Shirt (0.859)	Solid Men Polo Neck Black T-Shirt (0.711)	Printed Men Polo Neck Black T-Shirt (0.688)
women printed cotton kurta	Women Printed Cotton Blend Straight Kurta (0.627)	Women Printed Cotton Blend Straight Kurta (0.581)	Women Printed Cotton Blend Straight Kurta (0.581)
grey track pants	Striped Men Grey Track Pants (0.899)	Solid Women Grey Track Pants (0.865)	Solid Men Grey Track Pants (0.865)
men denim jacket	Full Sleeve Washed Men Denim Jacket (0.920)	Full Sleeve Washed Men Denim Jacket (0.920)	Full Sleeve Solid Men Denim Jacket (0.895)
women yoga pants	Solid Women Black Track Pants (0.253)	Solid Women Black Track Pants (0.242)	Solid Women Black, Dark Blue Track Pants (0.238)

The full top 10 of each query is in the notebook. Figure 2 shows the full top 10 of the hardest query.

Top 10 for 'women yoga pants' (19 AND results)

AND search + TF-IDF cosine ranking

Rank	pid	Score	Title
1	TKPFYG7THT4SKNEA	0.253	Solid Women Black Track Pants
2	TKPFYCGPENXCPBYK	0.242	Solid Women Black Track Pants
3	TKPFY485NTJM8AFP	0.238	Solid Women Black, Dark Blue Track Pants
4	TKPFY485BVMHNVFS	0.238	Solid Women Black, Dark Blue Track Pants
5	TKPFWV6JX5UBSUGV	0.208	Solid Women Black Track Pants
6	TKPFWV6JRM6MBEXH	0.206	Solid Women Dark Blue Track Pants
7	TKPFWV6JCPGUZEBG	0.206	Solid Women Grey Track Pants
8	TKPFWGHYPNQYZNFW	0.181	Solid Women Blue Track Pants
9	TKPFZMUZRZFGZMKK	0.175	Solid Women Black Track Pants
10	TKPFZMVF5B2GKFNT	0.175	Solid Women Black Track Pants

Figure 2. Top 10 for *women yoga pants*. All results are track pants whose description mentions yoga; the scores (0.18–0.25) are far lower than for the other queries (0.6–0.9) because *yoga* appears only once in a long description.

2.1 Evaluation metrics

All per-query metrics take the same inputs: `y_true` (label of each judged document, 1 = relevant, 0 = not relevant), `y_score` (the score our engine gives to that document) and the cut-off `k`. Here rel_i is the label of the document at position i of our ranking, R the number of relevant judged documents and Q the set of queries.

$$P@K = \frac{1}{K}\sum_{i=1}^{K} rel_i \qquad R@K = \frac{1}{R}\sum_{i=1}^{K} rel_i \qquad F1@K = \frac{2 \cdot P@K \cdot R@K}{P@K + R@K}$$

$$AP@K = \frac{\sum_{i=1}^{K} P@i \cdot rel_i}{\sum_{i=1}^{K} rel_i} \qquad MAP = \frac{1}{|Q|}\sum_{q \in Q} AP@K(q) \qquad MRR = \frac{1}{|Q|}\sum_{q \in Q} \frac{1}{rank_q}$$

$$DCG@K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)} \qquad NDCG@K = \frac{DCG@K}{IDCG@K}$$

Implementation decisions.

- **Documents not returned by the AND search** get score `NaN`. They never enter the top K, but they still count as relevant documents in the denominator of recall and in the ideal ranking of NDCG. Otherwise the AND filter would hide its own misses.
- **Ties** are broken with a stable sort, so equal scores keep the original order of the labels file and results are reproducible.
- **AP@K** divides by the number of relevant documents found in the top K. If none is found, AP@K = 0.
- **MRR:** $rank_q$ is the position of the first relevant document in the top K; if there is none, $1/rank_q = 0$.
- **IDCG** is the DCG of all judged documents sorted by their true label.

We checked the functions on a toy example with 5 judged documents (Figure 3). The values match our hand calculation.

Check of the metrics on a toy example (K = 3)

Labels 1, 0, 1, 1, 0 | scores 0.9, 0.8, 0.4, not retrieved, 0.1

Metric	Value
P@3	0.667
R@3	0.667
AP@3	0.833
F1@3	0.667
RR@3	1.000
NDCG@3	0.704

Figure 3. Metrics at K = 3 for labels 1, 0, 1, 1, 0 and scores 0.9, 0.8, 0.4, not retrieved, 0.1.

2.2 Results on the validation queries

Query	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
1: women full sleeve sweatshirt cotton	1.000	0.769	1.000	0.870	1.000	1.000
2: men slim jeans blue	0.600	0.600	0.915	0.600	1.000	0.706

MAP@10	MRR@10
0.958	1.000

2.3a Ground truth for the test queries

We acted as expert judges for our five test queries. A product is **relevant (1)** if a user typing the query would want to buy it: it must be the requested product type and match the gender, colour, fabric or use asked for. Otherwise it is **not relevant (0)**. For example, for *women yoga pants* we accepted track pants whose description says they are suitable for yoga.

For each query we labelled the top 20 products of our AND + TF-IDF ranking (19 for *women yoga pants*, whose AND search returns only 19 products): 99 judged products in total, stored in `custom_query_labels.csv`.

Query	Judged	Relevant	Not relevant
men black polo shirt	20	19	1
women printed cotton kurta	20	20	0
grey track pants	20	20	0
men denim jacket	20	20	0
women yoga pants	19	19	0

Metrics of the test queries at K = 10.

Query	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
men black polo shirt	1.000	0.526	1.000	0.690	1.000	1.000
women printed cotton kurta	1.000	0.500	1.000	0.667	1.000	1.000
grey track pants	1.000	0.500	1.000	0.667	1.000	1.000
men denim jacket	1.000	0.500	1.000	0.667	1.000	1.000
women yoga pants	1.000	0.526	1.000	0.690	1.000	1.000

MAP@10	MRR@10
1.000	1.000

2.3b Comments on the metrics and analysis of the results

The search engine was evaluated using five queries, with up to 20 retrieved documents manually labeled for each query at K = 10.

- **Precision@10** measures the proportion of relevant documents among the first ten retrieved results. All five queries achieved P@10 = 1.0, indicating that every product in the top ten was labeled relevant.
- **Recall@10** measures the proportion of known relevant documents appearing within the first ten positions. The results ranged from 0.500 to 0.526, meaning that approximately half of the relevant documents in the evaluated pool appeared in the top ten. This result is influenced by the evaluation methodology: only the top 20 retrieved documents were manually labeled, so recall does not represent coverage of the entire collection.
- **F1@10** combines precision and recall using their harmonic mean. The values ranged from 0.667 to 0.690. Although precision was perfect, F1 was lower because only approximately half of the judged relevant documents appeared within the first ten positions.
- **Average Precision@10 (AP@10)** evaluates the positions of relevant documents within the ranking. Every query achieved AP@10 = 1.0, indicating that all evaluated positions within the top ten contained relevant documents. However, the implementation averages

precision at relevant positions found within the cutoff, so this score should not be interpreted as evidence of complete retrieval.

- **Mean Average Precision@10 (MAP@10)** summarizes average precision across all queries. The system achieved MAP@10 = 1.0, indicating consistently high scores under the implemented evaluation method.
- **Reciprocal Rank@10 (RR@10)** measures the position of the first relevant document. Every query achieved RR@10 = 1.0, meaning that the first retrieved document was relevant in all five cases.
- **Mean Reciprocal Rank@10 (MRR@10)** averages reciprocal rank across the queries. The system achieved MRR@10 = 1.0, indicating that users could find at least one relevant product immediately for every evaluated query.
- **NDCG@10** evaluates ranking quality by rewarding relevant documents appearing at higher positions. All queries achieved NDCG@10 = 1.0, meaning that the top ten results matched an ideal ranking according to the binary relevance judgments.

The results indicate excellent precision for the evaluated queries. Every product appearing within the first ten positions was considered relevant, and the first result was relevant in every case. However, the perfect ranking metrics should be interpreted cautiously. If all retrieved documents in the evaluation pool are relevant, the evaluation cannot meaningfully distinguish between different rankings of those documents. Furthermore, the use of strict AND retrieval and a relevance pool constructed only from retrieved results means that the evaluation does not measure relevant products excluded from the search results. Therefore, the results demonstrate high precision for the selected queries, but they do not establish that the search engine achieves equally strong performance across the entire collection or for more difficult queries.

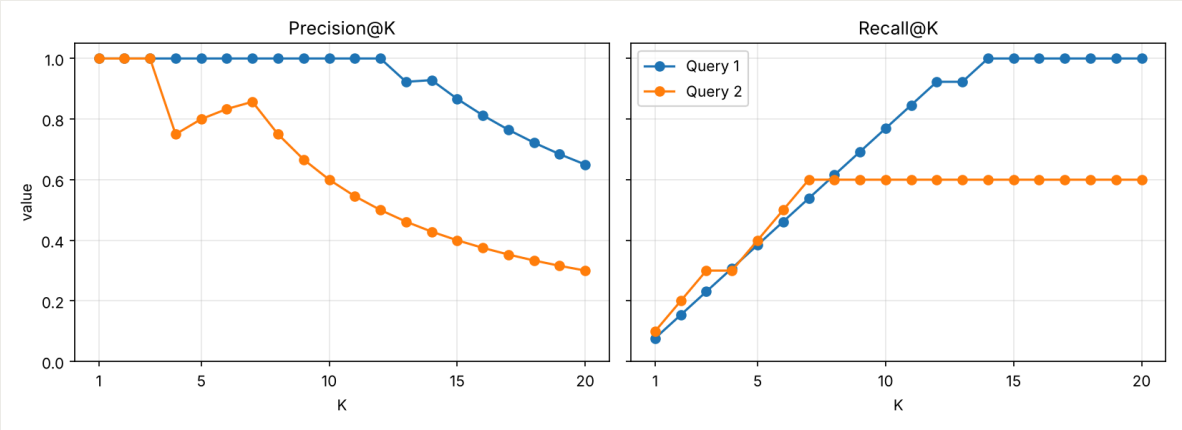


Figure 4. Precision@K and Recall@K for K = 1…20 on the two validation queries. The recall of query 2 stops at 0.6 from K = 7: the remaining relevant products are not returned by the AND search.

Validation queries. Query 1 reaches P@10 = 1.000, but the diagnostics of Section 2.4 show that none of the 10 products our engine really ranks first is in the official judgments, and 5 of them are men's sweatshirts. The validation metrics therefore only measure how we order the 20 judged products, not our real top 10. Query 2 is harder: the AND search returns only 8 of its 20 judged products and loses 4 relevant ones, so its recall stops at 0.600 from K = 7 (Figure 4) and P@10 = 0.600.

2.3c Limitations of the current system and proposed solutions

Limitation	Possible improvement
Strict AND matching, which may exclude relevant products (Figure 5)	OR-based retrieval
TF-IDF ranking without field-specific weights	BM25 or field-weighted ranking
Limited handling of synonyms	Query expansion
Limited handling of structured product attributes	Separate indexes for attributes such as color, gender and material
Gender matched outside the title: a *women* query returns 5 men's sweatshirts in its top 10 (Table 1)	Field weighting (title > details > description) or a gender filter
Near-duplicate products fill the top 10: the same title appears 7 times for *men denim jacket* (Table 2)	Group or diversify products with the same title
Noisy vocabulary: 43.4% of the terms are style or size codes and 64.0% appear in only one product (Table 3)	Drop keys such as *Style Code* and filter tokens mixing letters and digits
Brand is not indexed: *ECKO Unl men* returns 0 results although the brand has 951 products (Table 4)	Index brand as a separate field
Tokenization of compound words: *t-shirt* and *tshirt* give 1,814 vs 550 results (Table 4)	Normalise hyphenated and joined forms before indexing

Relevant products for 'men slim jeans blue' that the AND search does not return

4 of the 10 relevant validation products are missed because one query term is absent

pid	Title	Missing query term
JEAF8CHSVCP5GUH9	Slim Women Blue Jeans	men
JEAFUZXRDYSVFK4Y	Super Skinny Women Blue Jeans	men
JEAFUPSWVCTKXMHF	Tapered Fit Men Blue Jeans	slim
JEAFUZXSMRFFNGC2	Super Skinny Women Blue Jeans	men

Figure 5. Relevant validation products for *men slim jeans blue* that the AND search does not return, and the query term each one lacks.

2.4 Diagnostics

Section 2.4 of the notebook does not change the search engine; it produces the evidence used in 2.3b and 2.3c.

Table 1. Validation queries: what the metrics see and what they miss.

Query	AND results	Judged returned by AND	Judged in our real top 10	Relevant lost by AND	Other observation
1: women full sleeve sweatshirt cotton	500	20 / 20	0	0	5 men's sweatshirts in the top 10 (ranks 3, 5, 6, 7, 9)
2: men slim jeans blue	222	8 / 20	2	4	Missing term: *men* (3 products), *slim* (1)

Table 2. Near-duplicates in the top 10.

Query	Different titles in top 10	Most repeated title	Times
men denim jacket	3	Full Sleeve Solid Men Denim Jacket	7
men black polo shirt	3	Solid Men Polo Neck Black T-Shirt	6
men slim jeans blue	3	Super Skinny Men Blue Jeans	6
women printed cotton kurta	6	Women Printed Cotton Blend Straight Kurta (Orange)	5
women yoga pants	5	Solid Women Black Track Pants	5
women full sleeve sweatshirt cotton	3	Full Sleeve Solid Men Sweatshirt	5
grey track pants	6	Solid Women Grey, Blue Track Pants	3

Table 3. Noise in the vocabulary of the index.

Type of term	Terms	% of vocabulary
All terms	20,426	100.0%
Letter + digit codes (e.g. *3xl*, *style1005*, *38in*)	8,863	43.4%
Purely numeric	4,083	20.0%
Appearing in only one product (df = 1)	13,066	64.0%

Table 4. Query formulation: AND results for small wording changes.

Query	Results	Variant	Results
men black t-shirt	1,814	men black tshirt	550
women sweatshirt	685	women hoodie	323
men jeans	1,221	men denim	890
women pants	1,215	women trousers	1,155
men black polo shirt	229	men black polo shirt xl	45
ECKO Unl men (951 products of this brand)	0		

The notebook also shows, for the top 10 of each query, in which field (title, description or product details) each query term was matched. For *women yoga pants*, *yoga* appears only in the description or the product details, never in the title, which explains its low scores (0.18–0.25).

Reproducibility

Step-by-step instructions are in `project_progress/part_2/README.md`. In short:

1. Clone the repository and check out the tag: `git checkout IRWA-2026-part-2`.
2. Place `fashion_products_dataset.json` and `validation_labels.csv` in `data/` at the repository root (the folder is git-ignored because of the dataset size).
3. Create a virtual environment with Python 3.9 or later and run `pip install nltk pandas numpy matplotlib jupyter`.
4. Open `project_progress/part_2/02_indexing_evaluation.ipynb` and run all cells in order. The full run takes about one minute.

The search functions take the query as a string, so any query can be tried with `show_top("<query>", k=10)`.

Use of AI

AI assistants (ChatGPT and Claude) were used to explain Python functions, debug the evaluation code and review the report. All the code was reviewed and understood by the group.